

Ingenieria del Software

Tema 6. Pruebas

TABLA DE CONTENIDOS

- 01** Verificación y Validación (V&V)
- 02** Pruebas Automatizadas
- 03** Pruebas Manuales

TABLA DE CONTENIDOS

- 01** **Verificación y Validación (V&V)**
- 02** **Pruebas Automatizadas**
- 03** **Pruebas Manuales**

Verificación y Validación (V&V)

¿Qué es V&V?

La Verificación y Validación (V&V) es el conjunto de procesos de comprobación y análisis que aseguran que el software que se desarrolla está acorde a su especificación y cumple las necesidades de los clientes.

- ❑ **Verificación.** ¿Estamos construyendo el producto correctamente?
- ❑ **Validación.** ¿Estamos construyendo el producto correcto?

“build the product right and build the right product”

¿Qué es V&V?

La Verificación y Validación (V&V) es el conjunto de procesos de comprobación y análisis que aseguran que el software que se desarrolla está acorde a su especificación y cumple las necesidades de los clientes.

- ❑ **Verificación.** ¿Estamos construyendo el producto correctamente?
- ❑ **Validación.** ¿Estamos construyendo el producto correcto?

“build the product right and build the right product”

Para validar que el producto suple realmente las necesidades de negocio

- ❑ En la visión predictiva es habitual el señalamiento en la planificación de hitos en los que se hace una demostración de lo construido.
- ❑ En la visión adaptativa el producto se despliega en el entorno de producción para ser usado. Es el cliente o el usuario líder quien periódicamente valida el producto desplegado usándolo en su día a día.



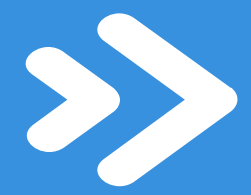
Clippy, el asistente para ayudar al usuario que Microsoft incluyó en la suite de productos Office 97 se construyó de forma impecable, sin embargo fue un producto fallido. ¿Realmente necesitaban los usuarios ayuda interactiva mientras usan el software?

¿Qué es V&V?

La Verificación y Validación (V&V) es el conjunto de procesos de comprobación y análisis que aseguran que el software que se desarrolla está acorde a su especificación y cumple las necesidades de los clientes.

- ❑ **Verificación.** ¿Estamos construyendo el producto correctamente?
- ❑ **Validación.** ¿Estamos construyendo el producto correcto?

“build the product right and build the right product”



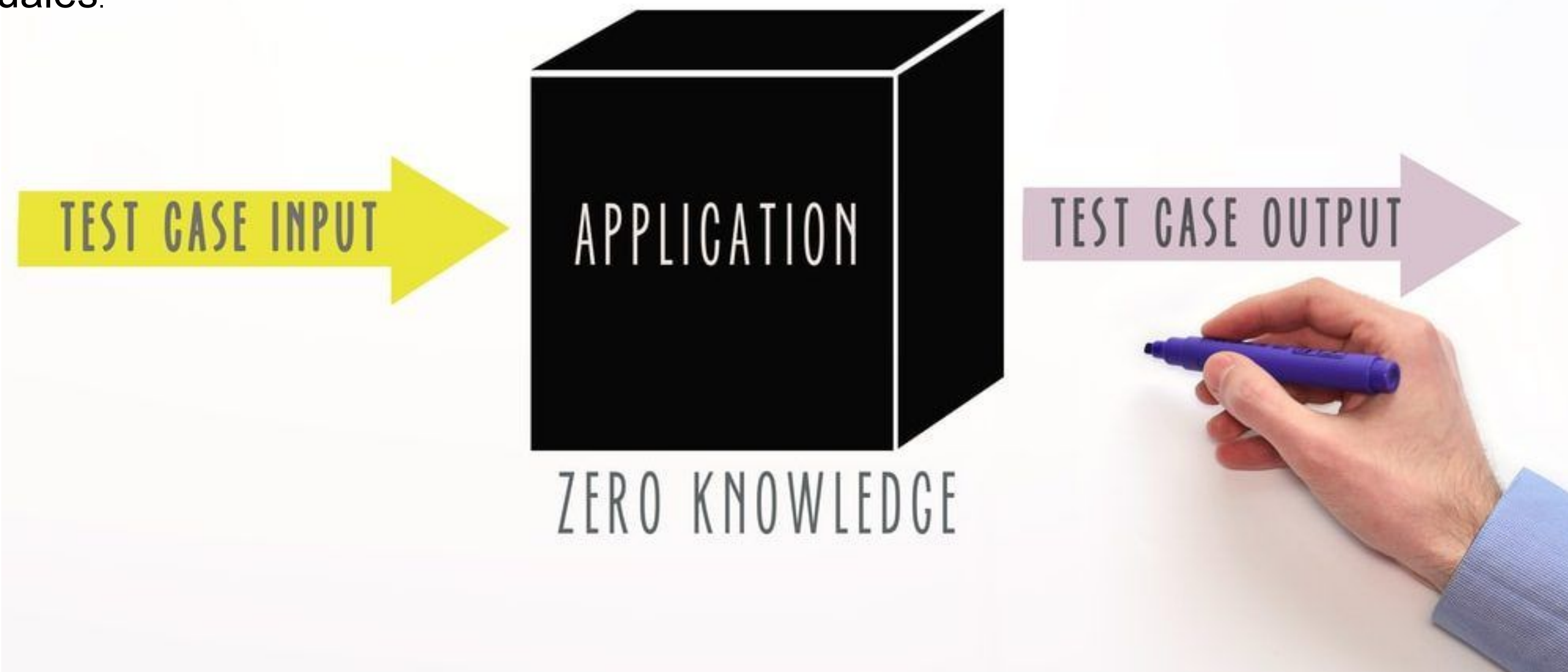
Verificación

Para **verificar que el producto se está construyendo sin fallos** se realizan pruebas.

- ❑ En la visión predictiva, las pruebas son realizadas por el equipo de QA o testers al finalizar la construcción.
- ❑ En la visión adaptativa, las pruebas son realizadas durante la construcción por el equipo de desarrollo. El objetivo del testing no solo es detectar fallos, sino en la medida de lo posible, prevenir dichos fallos.

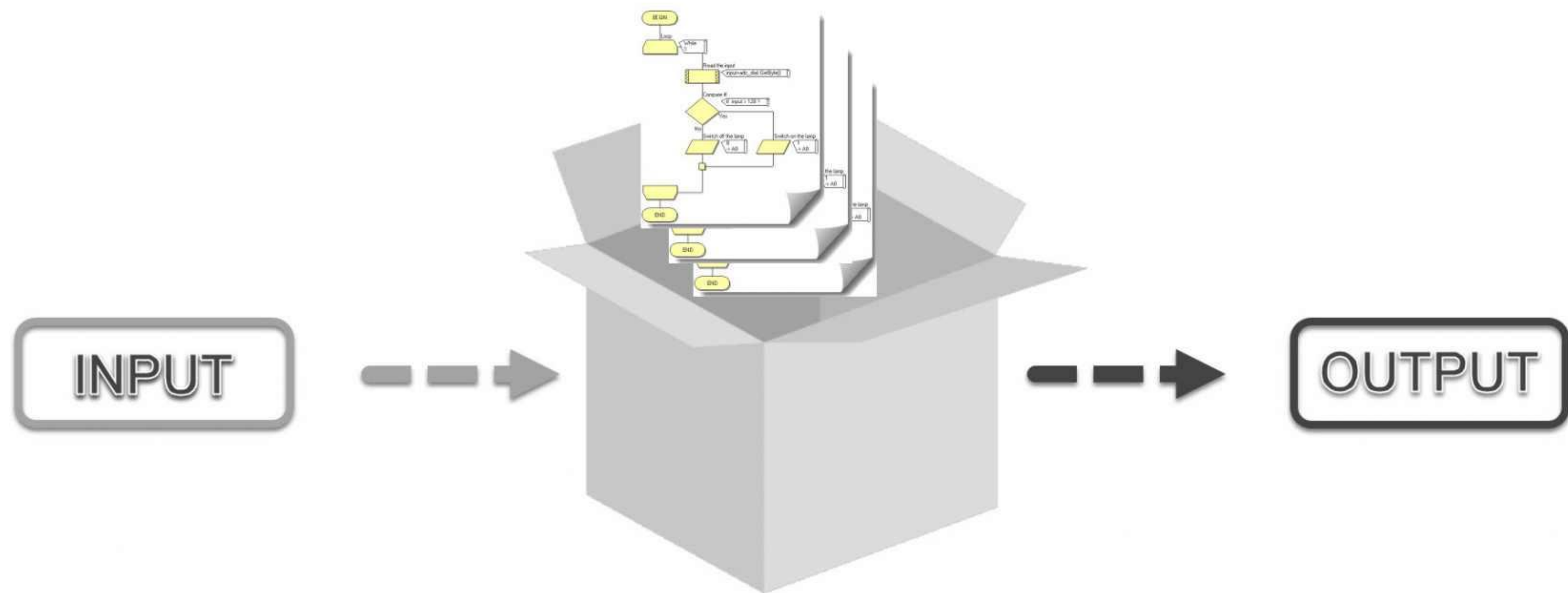
>> Verificación

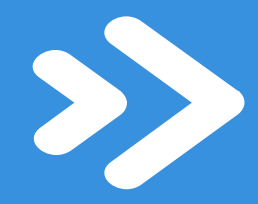
Las pruebas de “caja negra” centran su interés en lo que hace el componente no en cómo lo hace. Para ello el componente es alimentado con un conjunto de valores de entrada, la prueba certifica que los resultados devueltos por el componente son los que deben. Es indiferente el código ejecutado para lograr los resultados. Cada una de los pares “entrada” - “salida esperada” se llama caso de prueba o “*test case*”. Son las pruebas más habituales.



>> Verificación

Las pruebas de “caja blanca” no solo centran su interés en lo que hace el componente sino también en cómo lo hace. Para ello el componente es alimentado con un conjunto de valores de entrada y la prueba certifica que tanto los resultados devueltos por el componente como el código ejecutado es el que debe ser.





Verificación

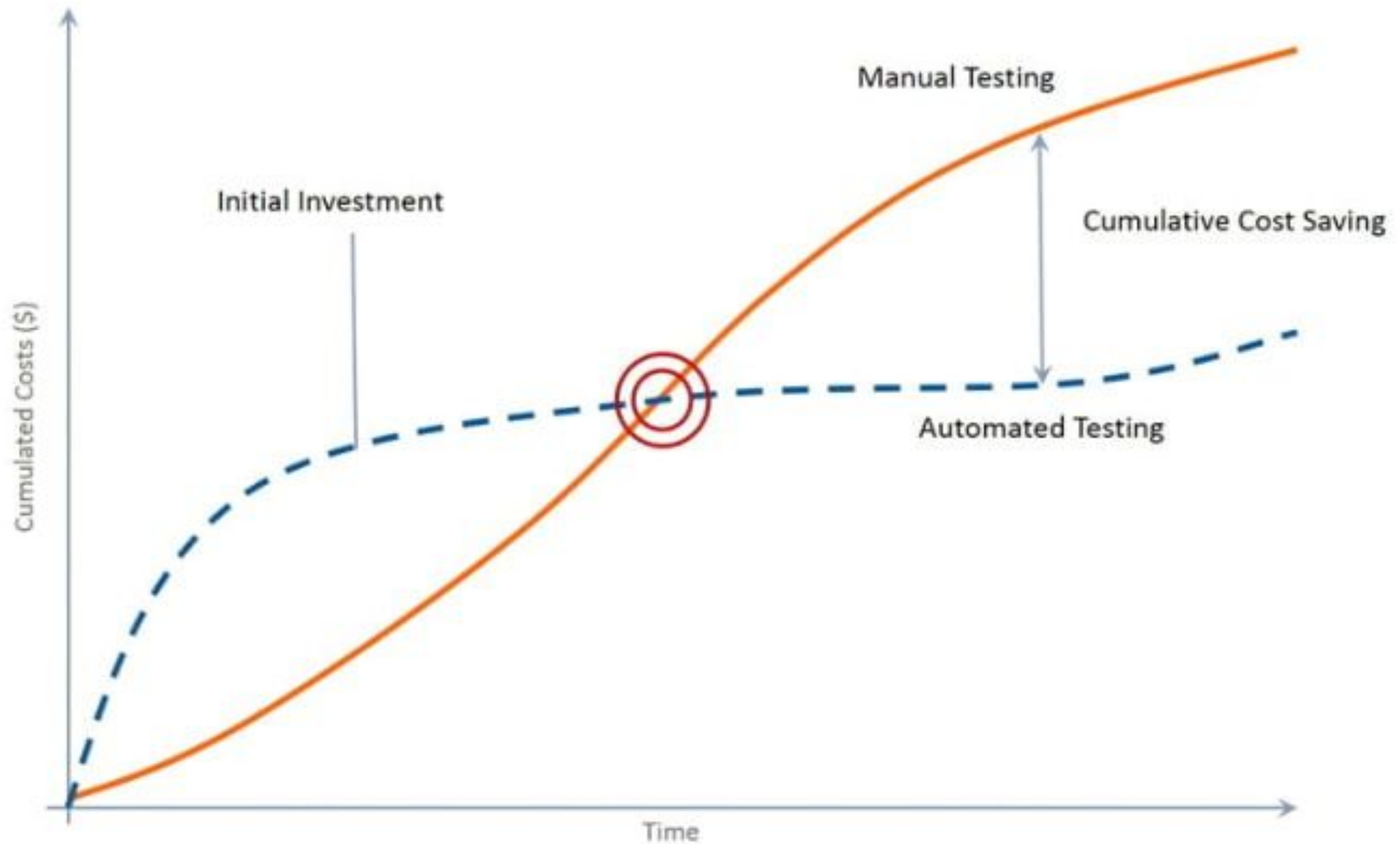


TABLA DE CONTENIDOS

- 01** Verificación y Validación (V&V)
- 02** Pruebas Automatizadas
- 03** Pruebas Manuales

Pruebas Automatizadas

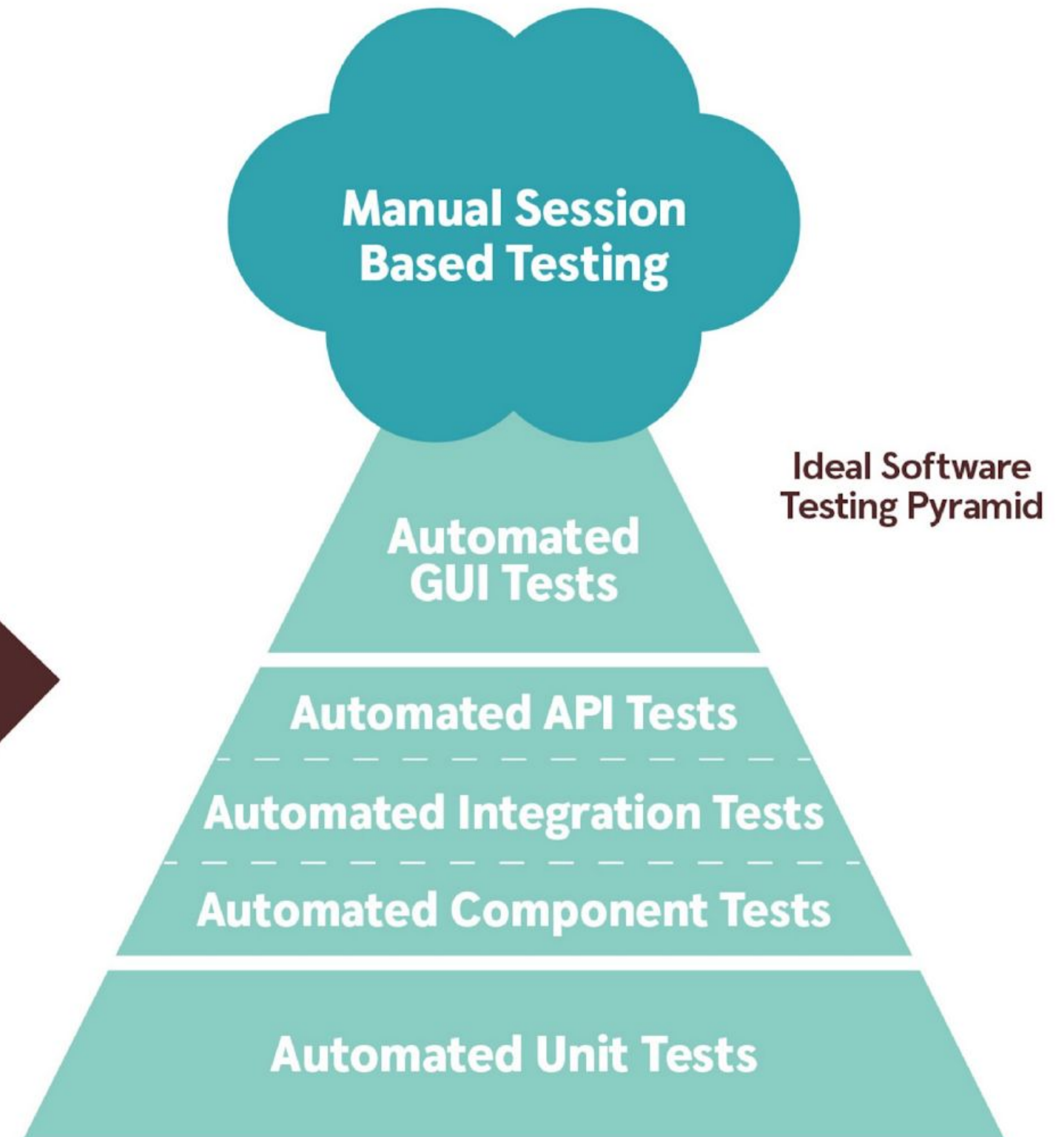
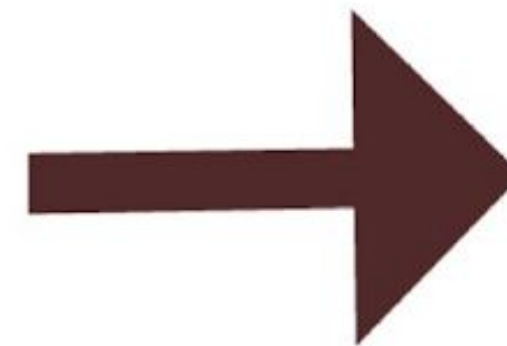
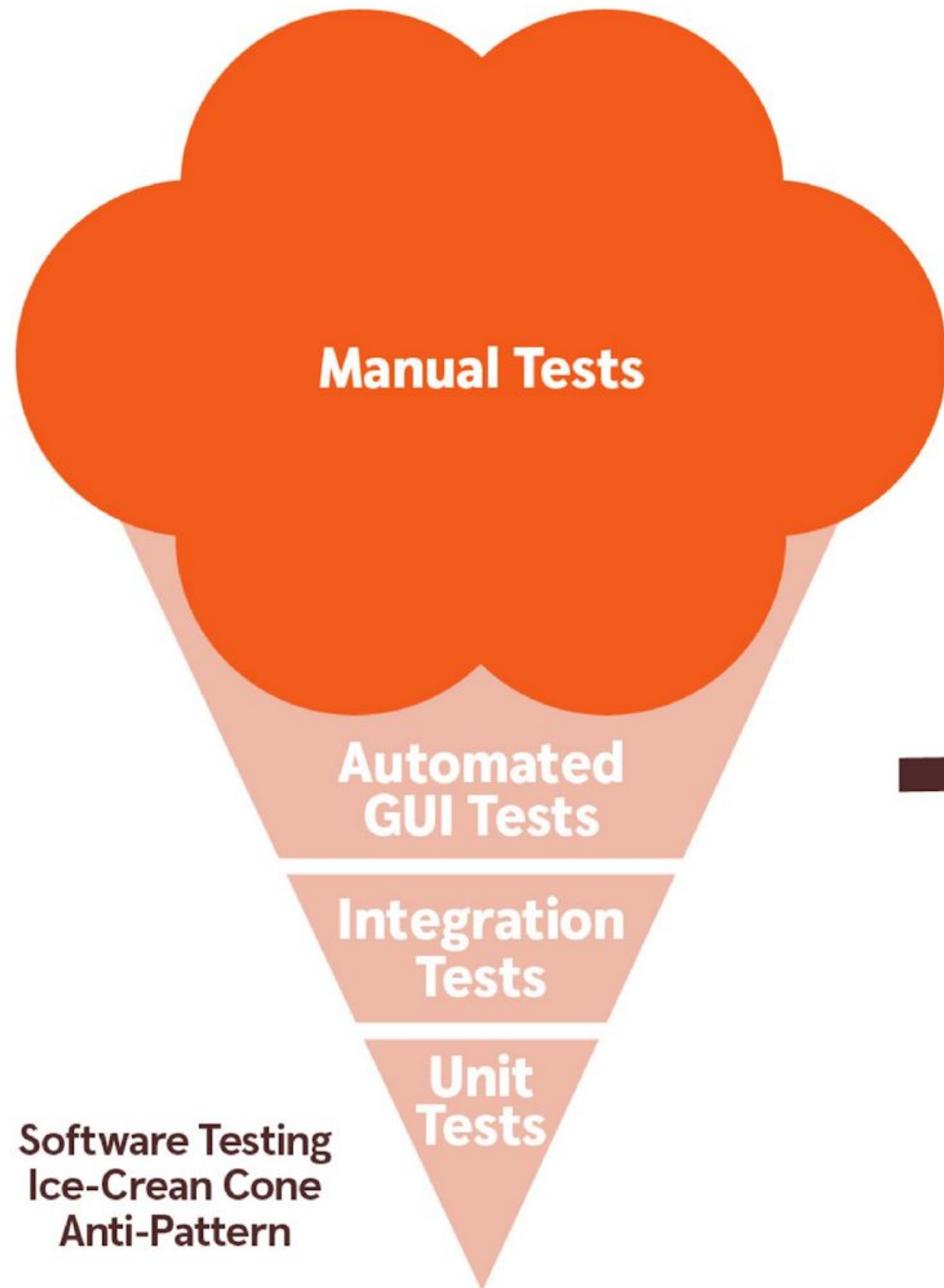
Tipos de Pruebas Automatizadas

- ❑ **Test unitario.** Prueba una única funcionalidad aislandola de todas sus dependencias. Esta prueba suele poner el foco en una única clase.
- ❑ **Test de integración.** Prueba que la interacción entre varios componentes funciona adecuadamente. Esta prueba suele requerir “ensamblar” una parte del producto instanciando varias clases (dependencias).
- ❑ **Test de aceptación o GUI.** Prueba todo el producto. Esta prueba requiere desplegar el producto completo.

Tipos de Pruebas Automatizadas

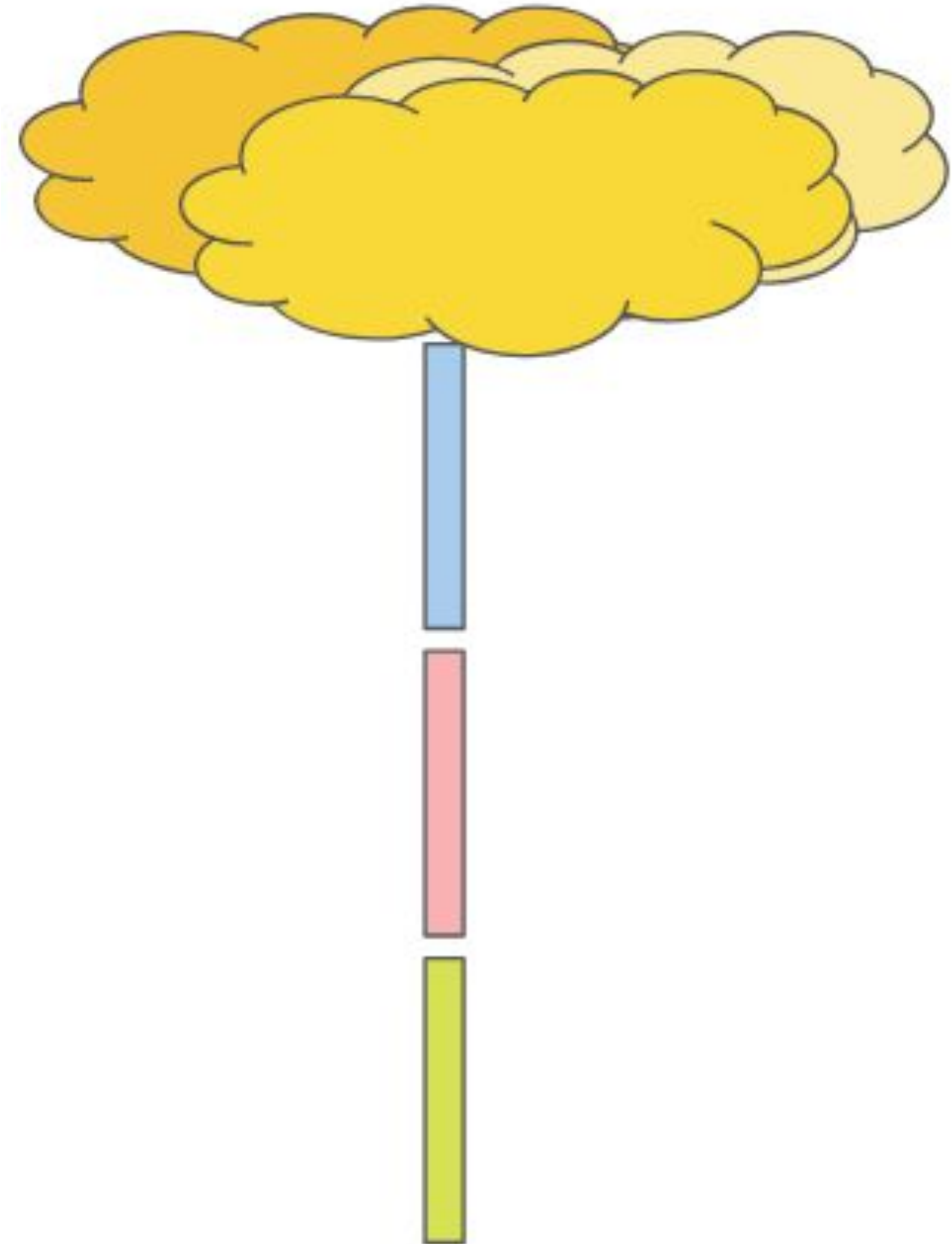
	Unitario	Integración	Aceptación
Tiempo de ejecución	<10 ms	<100 ms	< 10 s
Tamaño	Pequeño	Mediano	Grande
Fragilidad	Muy robusto	Frágil	Muy frágil
Repetibilidad	Muy repetible	Repetible	Repetible
Piezas ensambladas	Pocas	Muchas	Todas
Elementos de Infraestructura	Ninguno	Pocos	Muchos
Cantidad	Muchos	Algunos	Pocos

Tipos de Pruebas Automatizadas



Tipos de Pruebas Automatizadas

Quizás el antipatrón más extendido es el “cigarrillo”: muy pocos tests unitarios, muy pocos test de integración, muy pocos tests de aceptación y muchos tests manuales.



Tipos de Pruebas Automatizadas

- » **Tests Unitarios**
- » **Tests de Integración**
- » **Tests de Aceptación**

>> Test Unitario

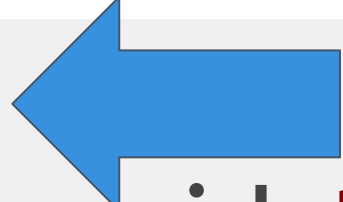
Las pruebas unitarias consisten en aislar una parte del código y comprobar que funciona como se espera. Son pequeños tests que validan el comportamiento de un objeto. Los encargados de escribir las pruebas son los desarrolladores. Los tests unitarios ayudan a documentar el código, a refactorizarlo, a detectar regresiones y a diseñarlo:

- ❑ **Emerger el diseño.** El programador se ve obligado a ejercitar su código. Al poner a prueba su implementación el programador sufre las consecuencias de su diseño aflorando oportunidades de mejora.
- ❑ **Documentar el código.** Los tests escritos facilitarán a otros programadores entender el propósito del código implementado. Es, además, una documentación viva, permanentemente actualizada.
- ❑ **Ayudan a refactorizar el código.** El programador cuenta con una red de seguridad que le permite limpiar el código haciéndolo más mantenible o extensible asegurándose que los cambios realizados no alterado el comportamiento esperado.
- ❑ **Permiten detectar regresiones o errores.** Si el programador introduce un error, será informado con una alerta al romperse algún test.

>> Frameworks

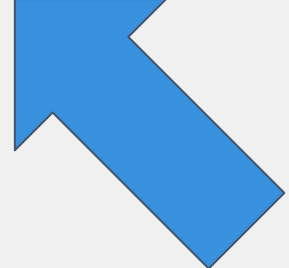
Existen abundantes frameworks (librerías) para ayudar al desarrollador a escribir las pruebas del código. Estos frameworks suelen acompañarse de plugins y extensiones para los principales IDEs de cara a facilitar tanto la codificación como la ejecución de las pruebas. Ejemplos de frameworks para testing unitario son: JUnit para java, Jasmine para javascript, Unittest para python, NUnit para C#, etc.

@Test



```
public void testAlumnoSeHaLicenciado() {  
    Alumno alumnoLicenciado = new Alumno("nombre_alumno", 5, 5);  
    assertTrue(alumnoLicenciado.seHaLicenciado());  
}
```

@Test



```
public void testAlumnoNoSeHaLicenciado() {  
    Alumno alumnoLicenciado = new Alumno("nombre_alumno", 5, 3);  
    assertFalse(alumnoLicenciado.seHaLicenciado());  
}
```

Para llevar a cabo buenas pruebas unitarias, deben seguir las tres A's del Unit Testing:

1. **Arrange** (organizar). Se establecen los valores de los parámetros de entrada y se instancia el objeto o sujeto a testar (SUT).
2. **Act** (actuar). Se desencadena el código objeto del test ejecutando un método del SUT.
3. **Assert** (afirmar). Se comprueba que el resultado obtenido es el esperado.

>> Test Unitario

@Test

```
public void Un_paciente_mayor_con_dos_sintomas_covid_requiere_aislamiento()
{
    // Dado un paciente de más de 70 años que tiene dos sintomas COVID19
    Paciente paciente = new Paciente();
    paciente.FechaNacimiento = LocalDate.of(1940,1,1);
    paciente.Sintomas = new String[]{"Tos seca", "Cansancio"};
    // Cuando chequeo si requiere o no aislamiento
    Boolean requiereAislamiento = paciente.RequiereAislamiento();
    // Entonces debe devolver cierto
    assertTrue(requiereAislamiento);
}
```

>> Test Unitario

Arrange (organizar). valores de entrada y sujeto a testar (SUT).

@Test

```
public void Un_paciente_mayor_con_dos_sintomas_covid_requiere_aislamiento()
{
    // Dado un paciente de más de 70 años que tiene dos sintomas COVID19
    Paciente paciente = new Paciente();
    paciente.FechaNacimiento = LocalDate.of(1940,1,1);
    paciente.Sintomas = new String[]{"Tos seca", "Cansancio"};
    // Cuando chequeo si requiere o no aislamiento
    Boolean requiereAislamiento = paciente.RequiereAislamiento();
    // Entonces debe devolver cierto
    assertTrue(requiereAislamiento);
}
```

>> Test Unitario

@Test

```
public void Un_paciente_mayor_con_dos_sintomas_covid_requiere_aislamiento()
{
    // Dado un paciente de más de
    Paciente paciente = new Paciente();
    paciente.FechaNacimiento = LocalDate.of(1940,1,1);
    paciente.Sintomas = new String[]{"Tos seca", "Cansancio"};

    // Cuando chequeo si requiere o no aislamiento
    Boolean requiereAislamiento = paciente.RequiereAislamiento();

    // Entonces debe devolver cierto
    assertTrue(requiereAislamiento);
}
```

Act (actuar). Desencadenar el código objeto del test ejecutando un método del SUT.

>> Test Unitario

@Test

```
public void Un_paciente_mayor_con_dos_sintomas_covid_requiere_aislamiento()
{
    // Dado un paciente de más de 70 años que tiene dos sintomas COVID19
    Paciente paciente = new Paciente(75, "Juan", "Pérez", "1945-01-01", "Masculino", "Español", "Cataluña", "Barcelona");
    paciente.FechaNacimiento = LocalDate.of(1945, 1, 1);
    paciente.Sintomas = new String[]{"Fiebre", "Tos seca", "Cansancio"};
    // Cuando chequeo si requiere o no aislamiento
    Boolean requiereAislamiento = paciente.RequiereAislamiento();

    // Entonces debe devolver cierto
    assertTrue(requiereAislamiento);
}
```

Assert (afirmar). Comprobación que el resultado obtenido es el esperado.





Características

F	Fast	Rápida ejecución. Unos pocos milisegundos.
I	Isolated	Independiente de otros test.
R	Repeatable	Se puede repetir en el tiempo.
S	Self-Validating	Cada test debe poder validar si es correcto o no a sí mismo.
T	Timely	Deben escribirse en el momento adecuado durante la codificación.

Un test unitario debe ser

- ❑ **rápido**, debe ejecutarse en unos pocos milisegundos,
- ❑ **robusto**, no debe romperse al modificar el código (no es frágil),
- ❑ **repetible**, una misma entrada debe obtener un mismo resultado en todo momento

Tipos de Pruebas Automatizadas

- » **Tests Unitarios**
- » **Tests de Integración**
- » **Tests de Aceptación**

>> Test Integración



dos tests unitarios, ningún test de integración

>> Test Integración

*tres tests unitarios,
ningún test de
integración*



>> Test Integración

En los tests de integración el sujeto bajo test (SUT) es la unión (cableado o ensamblaje) de diferentes componentes del producto. Es posible “ensamblar” un componente de infraestructura de este modo el test puede acceder a ficheros de texto, a base de datos, a servicios web, etc. El uso de elementos de infraestructura “externos” a las propias clases del core de negocio puede empeorar las características del test:

- ❑ **Lento.** Acceder al disco duro, consumir un web service o acceder a la base de datos puede llevar cientos de milisegundos.
- ❑ **Frágil.** Al ensamblar muchos componentes, una modificación en uno de ellos puede hacer que el tests se “rompa” (deje de funcionar).
- ❑ **No repetible.** El resultado del test (OK o NOK) puede depender del estado de los elementos de infraestructura (contenido de un fichero o columnas de un registro de base de datos)

>> Test Integración

@Test

```
public void testIngresar(){  
    String fichero = "paciente.txt"  
    String respuesta = new ServicioAdmision().ingresar(fichero);  
    AssertEquals(respuesta, "Navarro, Fermin AISLAMIENTO\n")  
}
```

@Test

```
public void testIngresarFicheroVacio(){  
    String fichero = "pacientesVacio.txt";  
    String respuesta = new ServicioAdmision().ingresar(fichero);  
    AssertEquals(respuesta, "")  
}
```

Tipos de Pruebas Automatizadas

- » **Tests Unitarios**
- » **Tests de Integración**
- » **Tests de Aceptación**

>> Test Aceptación

El objetivo de los tests de aceptación es automatizar las comprobaciones y validaciones que los testers manuales realizan interactuando con el interface gráfico del producto software y que han indicado es su plan de pruebas. El caso de prueba comienza siendo manual, para luego ser automatizado.

Estas pruebas son muy **lentas** (pueden llevar varios segundos o incluso minutos), son muy **frágiles** (cambiar el nombre del botón puede romper el test) y **poco ágiles** (cuando un tests se pone en “rojo”, puede resultar muy difícil determinar la causa del fallo).

Por otro lado, estos tests suelen requerir reescribirse con frecuencia, la interfaz de usuario es muy propensa a cambios.

>> Test Aceptación

Dada la vista de diagnósticos tras hacer login

Cuando busco por código 'aaaaaaaaa'

Entonces no se devuelve se muestra un aviso indicando que ningún diagnóstico coincide con los parámetros de búsqueda.

navarra.es Consultas Administración Admin, Admin Salir

Diagnósticos - Búsqueda de diagnósticos

Por diagnóstico Por servicio

Código: aaaaaaaaa

Descripción:

Tipo de garantía:

Limpiar Buscar

navarra.es Consultas Administración Admin, Admin Salir

Diagnósticos - Búsqueda de diagnósticos

No se han encontrado diagnósticos.

Por diagnóstico Por servicio

Código: aaaaaaaaa

Descripción:

Tipo de garantía:

Limpiar Buscar

>> Test Aceptación

Dada la vista de gestión de pacientes
Cuando busco sin introducir ningún criterio

Entonces se muestra un aviso
indicando que hay que indicar algún
criterio

Búsqueda

Paciente

CIPNA

Localización

Seleccione un centro

Limpiar

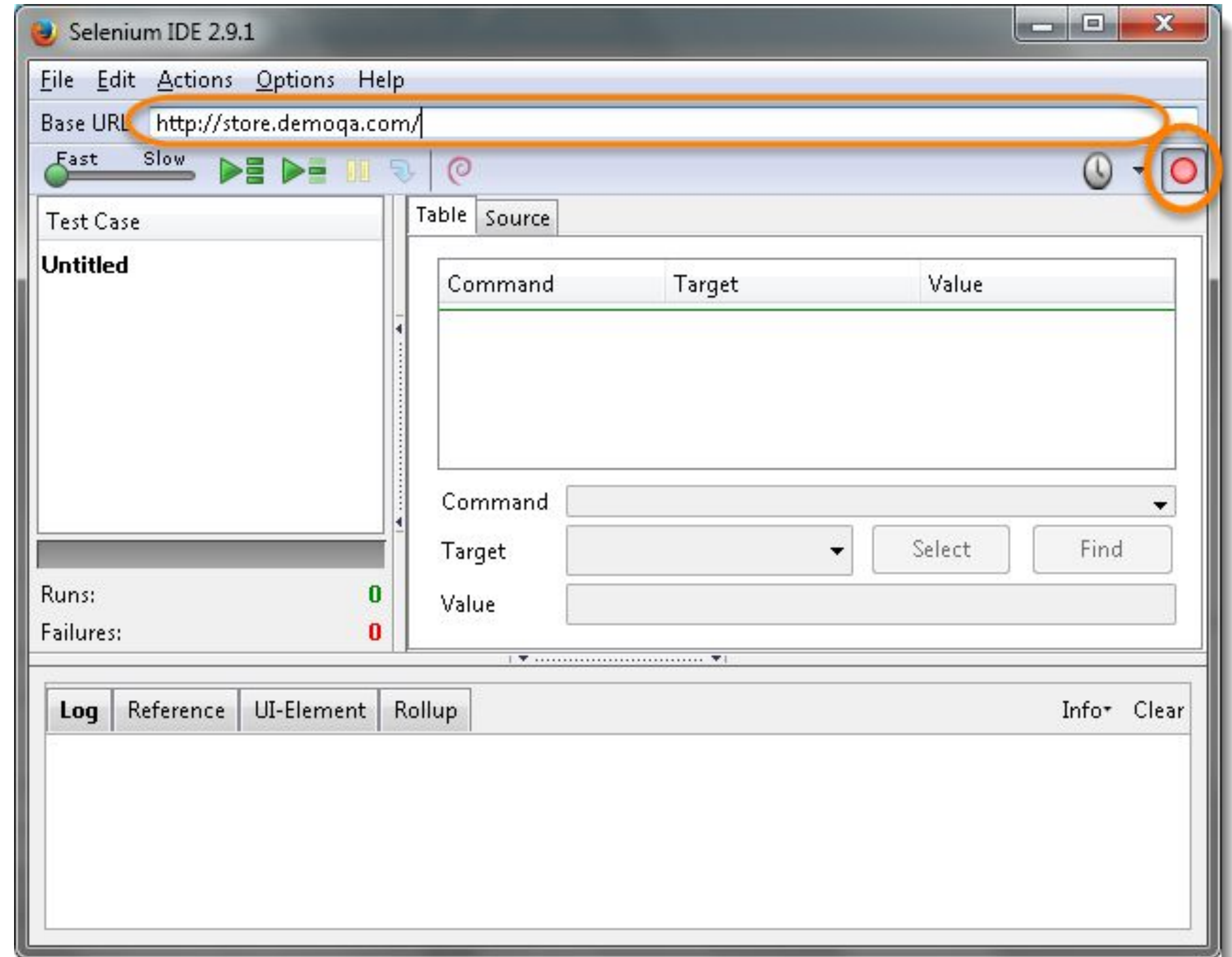
Buscar

Tiene que introducir filtros de búsqueda.

Búsqueda

>> Test Aceptación

Existen herramientas que permiten “grabar” la interacción del usuario con el producto. Una vez grabado el test la herramienta es capaz de reproducir la grabación moviendo el puntero y pulsando botones y teclas. Además es capaz de comparar la respuesta esperada con la obtenida. Una de las herramientas “record & play” más potentes es Selenium IDE.

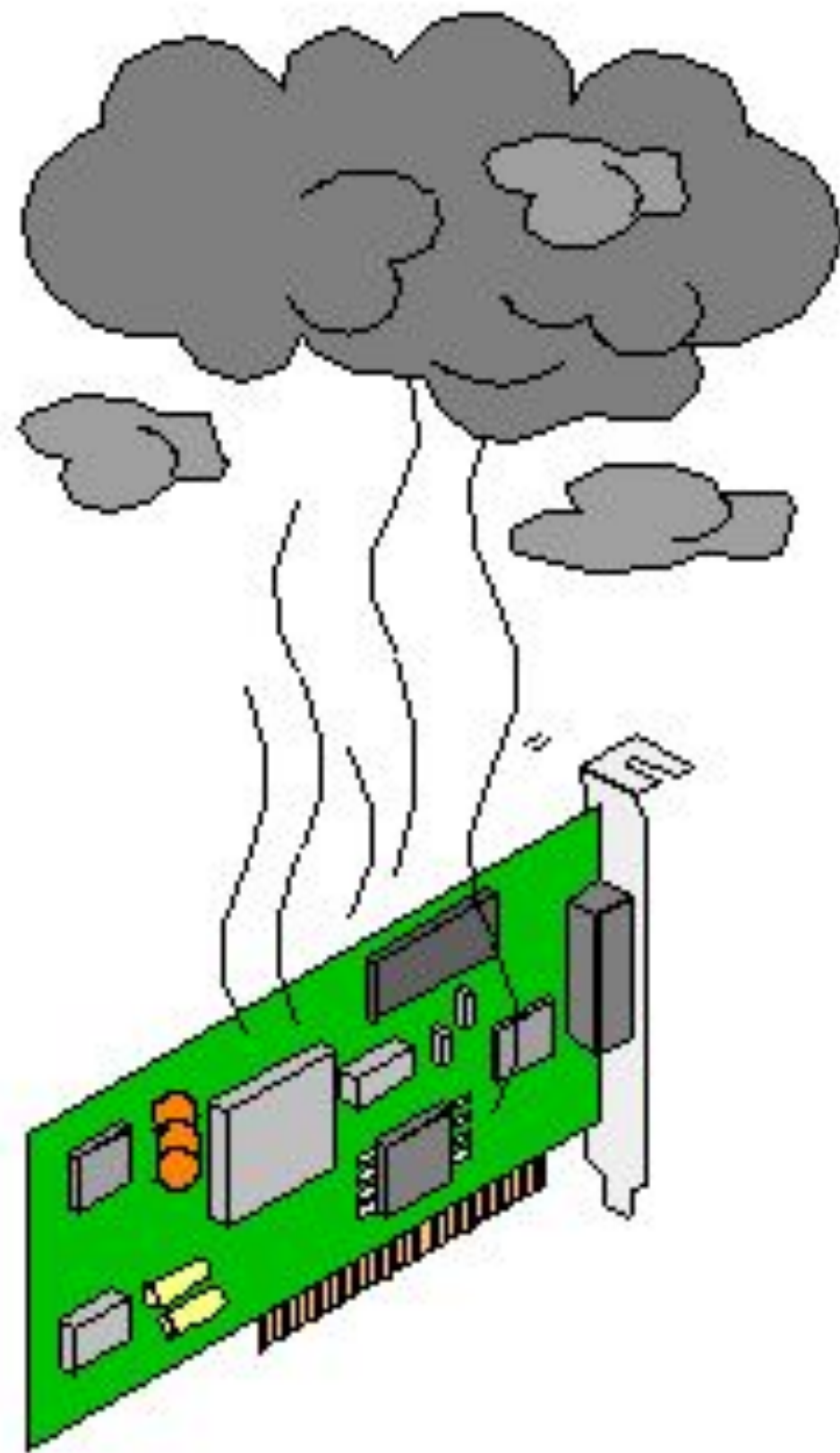


>> Test Aceptación

```
@Test
public void verifyLogin() throws InterruptedException{
    WebDriver driver = new FirefoxDriver();
    driver.get("http://www.gmail.com");
    driver.findElement(By.id("Email")).sendKeys("testmail@gmail.com");
    driver.findElement(By.id("next")).click();
    Thread.sleep(1000);
    driver.findElement(By.id("Passwd")).sendKeys("p4ssw0rd");
    driver.findElement(By.id("singIn")).click();
}
```

Otras herramientas ofrecen al desarrollador librerías y APIs que facilitan la interacción con el navegador. Estas herramientas permiten enviar pulsaciones de teclas o clicks del ratón al navegador de forma programática, además permiten inspeccionar la respuesta HTML obtenida. Un ejemplo son Selenium Web Driver o Protractor.

>> Test Aceptación



El test de humo o **smoke test** pretende asegurar que ciertas características básicas del producto funcionan perfectamente. Por ejemplo, se implementan smoke tests para probar que al cargar las diferentes pantallas en la consola del navegador no aparecen errores o para asegurar que al pedir una pantalla de acceso restringido el usuario es redirigido a la pantalla de login. Son tests superficiales que no prueban el sistema exhaustivamente, solo pretenden probar que el producto ha desplegado todas las piezas necesarias (por ejemplo la pantalla de login o los recursos). Las pruebas exhaustivas de cada pieza se realizan con tests unitarios y de integración.

TABLA DE CONTENIDOS

- 01** Verificación y Validación (V&V)
- 02** Pruebas Automatizadas
- 03** Pruebas Manuales

Pruebas Manuales

Pruebas Manuales

La automatización de algunas pruebas puede requerir un gran esfuerzo o simplemente ser imposible, por ejemplo, validaciones de aspectos cosméticos del interfaz de usuario son difícilmente implementables por máquinas. En estos casos es imprescindible la realización de pruebas manuales.

Estas pruebas manuales pueden ser llevadas a cabo o bien por el propio equipo de desarrollo en metodologías de construcción ágiles o bien por un equipo de especialistas habitualmente denominado QA (Quality Assurance, aseguramiento de la calidad).

U1	Formulario de entrada de datos
U1.1	Comprobar la correcta tabulación de los campos de la pantalla
U1.2	Entrar a la pantalla sin datos para mostrar
U1.3	Cambiar de páginas en gridview paginados y comprobar que cambian los registros que se muestran
U1.4	Hacer clic en botones de Modificar y borrar sin tener un registro seleccionado
U2	Alta de un registro
U2.1	Dar de alta un registro con datos correctos y comprobar que se guarda en la BBDD
U2.2	Introducir fechas incorrectas o nulas en campos de fecha
U2.3	Introducir caracteres, valores fuera de rangos y nulos en campos numéricos
U2.4	Comprobar que se ha fijado la longitud máxima de caracteres en campos de texto
U2.5	Dejar vacíos campos obligatorios
U2.6	Introducir valores clave repetidos (códigos, DNI, etc...)
U3	Baja de un registro
U3.1	Comprobar que si existen datos relacionados con el registro a borrar, también se borran, o se advierte para no ser borrado (dependiendo del uso)
U3.2	Borrar último registro de la base de datos y comprobar que no da errores.
U3.3	Borrar el primer registro
U4	Modificación de un registro
U4.1	Dar de alta un registro con datos correctos y comprobar que se guarda en la BBDD
U4.2	Introducir fechas incorrectas o nulas en campos de fecha
U4.3	Introducir caracteres, valores fuera de rangos y nulos en campos numéricos
U4.4	Comprobar que se ha fijado la longitud máxima de caracteres en campos de texto
U4.5	Dejar vacíos campos obligatorios
U4.6	Introducir valores clave repetidos (códigos, DNI, etc...)

U1	Formulario de entrada de datos
U1.1	Comprobar la correcta tabulación de los campos de la pantalla
U1.2	Entrar a la pantalla sin datos para mostrar
U1.3	Cambiar de páginas en gridview paginados y comprobar que cambian los registros que se muestran
U1.4	Hacer clic en botones de Modificar y borrar sin tener un registro seleccionado
U2	Alta de un registro
U2.1	Dar de alta un registro con datos correctos y comprobar que se guarda en la BBDD
U2.2	Introducir fechas incorrectas o nulas en campos de fecha
U2.3	Introducir caracteres, valores fuera de rangos y nulos en campos numéricos
U2.4	Comprobar que se ha fijado la longitud máxima de caracteres en campos de texto
U2.5	Dejar vacíos campos obligatorios
U2.6	Introducir valores clave repetidos (códigos, DNI, etc...)
U3	Baja de un registro
U3.1	Comprobar que si existen datos relacionados con el registro a borrar, también se borran, o se advierte para no ser borrado (dependiendo del uso)
U3.2	Borrar último registro de la base de datos y comprobar que no da errores.

Diseño de Casos de Prueba			Planificado	Probado			
ID Caso Prueba	Procedimiento/ Pasos de la Prueba	Datos de Entrada	Fecha	Responsable	Fecha	Resultado/ Estado	Observaciones/nº incidencia
I1	Hacer una reserva en la web y comprobar que abre el BIOMAX	Usuario, contraseña	13/12/06	Roberto	09/01/07	Correcto	
I2	Cambiar patrones horario y aplicación de los mismos y comprobar que cambia el horario de reserva		13/12/06	Luis	13/12/06	Correcto	
I3	Crear un usuario con horario especial y comprobar que abre en los momentos establecidos		09/01/07	Roberto	11/01/07	Correcto	